

LATVIJAS UNIVERSITĀTE
EKSAKTO ZINĀTŅU UN TEHNOLOĢIJU FAKULTĀTE
MATEMĀTIKAS NODAĻA

PERFEKTO POLIMONDU EKSISTENCE

KURSA DARBS

Autors: Marta Rudzāte

Studenta apliecības nr.: mr18118

Darba vadītājs: prof. Dr. math. Andrejs Cibulis

RĪGA 2025

ANOTĀCIJA

Darbs veltīts perfektu polimondu izpētei, galvenokārt apskatot eksistences jautājumu. Līdz šim eksistence perfektiem polimondiem ar nepāra malu skaitu ir ļoti maz aprakstīta, tāpēc tiek uzrādītas vairākas konstrukcijas šiem polimondiem. Šīs konstrukcijas ir aprakstītas ar paralēlām bezkonteksta gramatikām. Darba gaitā arī tika izveidotas datorprogrammas, kas spēj atrast visus perfektos polimondus dotam malu skaitam.

Atslēgas vārdi: izogoni, goligoni, polimino, polimondi, bezkonteksta gramatika, paralēlā bezkonteksta gramatika.

Keywords: isogons, golygons, polyomino, polyiamonds, context-free grammar, parallel context-free grammar.

SATURA RĀDĪTĀJS

DEFINĪCIJAS

- 1. definīcija.** Par polimino (angl. *polyomino*) sauc plaknes figūru, kas sastāv no vienības kvadrātiem, kuri pievienoti viens otram pa vesela garuma malām.
- 2. definīcija.** Par polimondū (angl. *polyiamond*) sauc plaknes figūru, kas sastāv no regulāriem vienības trijstūriem, kuri pievienoti viens otram pa vesela garuma malām.
- 3. definīcija.** Ja daudzstūra (polimino vai polimonda) visi malu garumi, sākot ar vienu virsotni, iet pēc kārtas augošā secībā no 1 līdz n , tad tādu daudzstūri sauc par perfektu.
- 4. definīcija.** Par 90 grādu secīgu izogonu (angl. *serial isogon of 90 degrees*) sauc daudzstūri, kurš satur tikai taisnus leņķus, un kura malu garumi, sākot no kādas virsotnes, ir naturāli skaitļi augošā secībā no 1 līdz n . Šādu daudzstūri sauc arī par goligonu (angl. *golygon*).

IEVADS

Kursa darbā tiek aplūkoti perfektie polimondi, par kuriem zinātniskajā literatūrā ir pieejama ļoti ierobežota informācija. 1988. gadā Lī Salovs (*Lee Sallows*) definēja 90 grādu secīgu izogonu. Visi polimino ir arī izogoni, tomēr ne visi izogoni ir polimino. Polimino klasē tiek pētīti vairāki saturīgi uzdevumi, kādu nav izogonu klasē, piemēram, jautājums par maksimālo un minimālo laukumu, kā arī, vai ar polimino ir iespējams pārklāt plakni.

Pirmais jautājums, kurš rodas pēc perfektu polimino un polimondu definēšanas, ir to eksistence jebkuram malu skaitam. Atbildi uz šo jautājumu ir daļēji devuši Lī Salovs kopā ar Martinu Garneru (*Martin Gardner*), uzrādot konstrukciju perfektiem polimino. Latvijas Universitātes pasniedzēja Elīna Buliņa savā bakalaura un maģistra darbā ir uzrādījusi konstrukcijas perfektiem polimondiem, kuru malu skaitu var izteikt ar $8k$ un $8k - 2$. Perfektu polimondu konstrukcija katram pāra malu skaitam $n \geq 6$ ir atrodama rakstā [**cibulis_bulina**].

Darba mērķis ir pierādīt eksistenci perfektiem polimondiem ar nepāra malu skaitu $n \geq 5$. Mērķa sasniegšanai tika izvirzīti šādi darba uzdevumi:

1. iepazīties ar literatūru, kas pieejama par perfektiem polimino un polimondiem;
2. uzrakstīt datorprogrammu, kas izveido visus perfektos polimondus dotam malu skaitam;
3. atrast konstrukcijas, kas veido perfektos polimondus;
4. aprakstīt konstrukcijas ar bezkonteksta un paralēlām bezkonteksta gramatikām;
5. pierādīt konstrukciju korektumu.

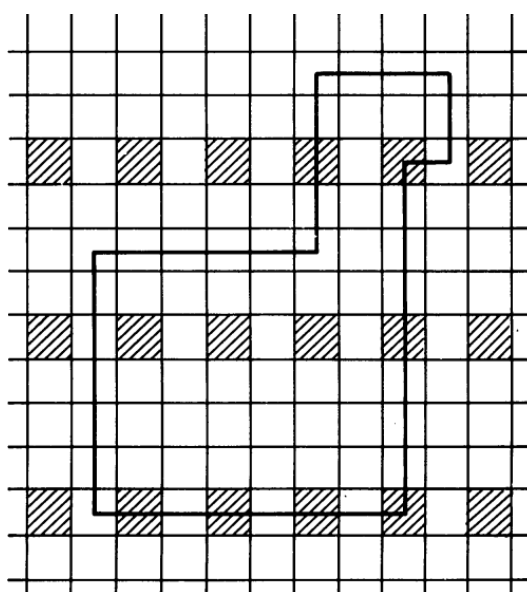
Darbs sastāv no trīs nodaļām, definīciju saraksta un diviem pielikumiem. Pirmā nodaļa veltīta pāra skaita malu figūrām: izogoniem, polimino un polimondiem. Otrā nodaļa veltīta polimondu aprakstīšanai ar bezkonteksta gramatikām, bet trešā nodaļa – polimondu aprakstīšanai ar paralēlām bezkonteksta gramatikām. Darbā iekļauta 1 tabula, 23 attēli, 2 pielikumi un 8 literatūras avoti.

1. PĀRA SKAITA MALU FIGŪRAS

1.1. Izogoni un goligoni

Rakstā [sallows1991serial] L. Salovs definēja 90 grādu secīgu izogonu kā ceļu, kurš sākas ar vienu garuma vienību, pēc tam pagriežas 90° jebkurā virzienā un iet divas vienības uz priekšu, tad atkal pagriežas 90° jebkurā virzienā un tagad jau iet trīs vienības uz priekšu, un tā tālāk. Beigās ir jāatgriežas sākumpunktā, veidojot taisnu leņķi ar pirmo posmu. Izogonam ir atļauts sevi krustot, pieskarties stūros un pārklāties posmos. Šajā rakstā ir arī aprakstītas un pamatotas vairākas interesantas lietas par šiem izogoniem.

Algebriski ir pamatots, ka 90° secīgam izogonam malu skaitam ir jābūt $8k$, kur $k \geq 1$. To pašu M. Gārners pamato vizuāli ar specifiski iekrāsotu rūtiņu režģi. Metodes ideju var redzēt 1. attēlā, kur jau ir iezīmēts izogons ar 8 malām. Izogona ceļš jāsāk horizontāli no iekrāsotas rūtiņas. Izogons atkārtoti (virziena izvēles brīdī) nokļūs iekrāsotā rūtiņā, kas ir vertikāli sākuma iekrāsotajai rūtiņai, tad un tikai tad, ja malu skaits dalīsies ar 8. Lai pamatotu, ka katram malu skaitam $8k$ eksistē izogons, ir dota konstrukcija. Šī konstrukcija sevi vienmēr krustos, kad $k > 1$. Bet ir dota arī otra konstrukcija, kas veido polimino: Pirmajos $N/4$ posmos iet tikai austrumu un ziemeļu virzienā (pirmajā posmā iet uz austrumiem), tad nākamajos $N/2$ posmos iet tikai rietumu un dienvidu virzienā, atlikušos $N/4$ posmos atkal iet tikai austrumu un ziemeļu virzienā. 1. attēlā ir iezīmēts polimonds, kurš atbilst šai konstrukcijai, kad $N = 8$, bet 2. attēlā, kad $N = 16$.



1. att. Iekrāsots rūtiņu režģis ar izogonu

Donands Knuts (*Donald Knuth*) šajā pašā rakstā ir devis procedūru, ar kuras palīdzību var saskaitīt, cik izogonu eksistē, ja fiksē tā posmu skaitu. Šī procedūra ir nedaudz piņķerīga, tāpēc Ričards Kenets Gajs (*Richard Kenneth Guy*) ir devis formulu, kurā vien ievietojot malu skaitu, uzreiz iegūst atbildi, tomēr tā ir tikai aptuvena.

L. Salovs ir arī pavisam nedaudz aplūkojis perfektus polimondus: ir pateikts, ka N -stūru polimondi, kuri satur tikai 60 un 300 grādu leņķus, eksistē, ja $N > 9$ un $N \not\equiv 1 \pmod{3}$, bet polimondi, kas satur tikai 120 un 240 grādu leņķus, eksistē, ja $N = 6n$.

1990.gada rakstā [**dewdney1990odd**] Aleksandrs Djūdņijs (*Alexander Dewdney*) 90 grādu secīgu izogonu pārdēvē par goligonu un citiem vārdiem apraksta rakstā [**sallows1991serial**] dotos pierādījumus.

1.2. Polimino un polimondi

Elīnas Buliņas bakalaurs darbā [**bulicna2016magiskie**] un Valmieras 11. Studentu konferences rakstu krājumā [**bulicna2017magiskie**] ir detalizēti aprakstīta metode, kā konstruēt perfektu $8k$ -stūru polimino, kur $k \geq 1$, kā arī ir pierādīts, ka šī konstrukcija vienmēr dos slēgtu figūru. Šī ir tā pati konstrukcija, kura tika aprakstīta 1991.gada rakstā [**sallows1991serial**], vien polimondi ir apgriezti spoguļattēlā un par 90° pagriezti pulksteņa rādītāja kustības virzienā.

Polimino veidošanas shēma izskatās šādi:

1. izvēlas $n = 8k$,

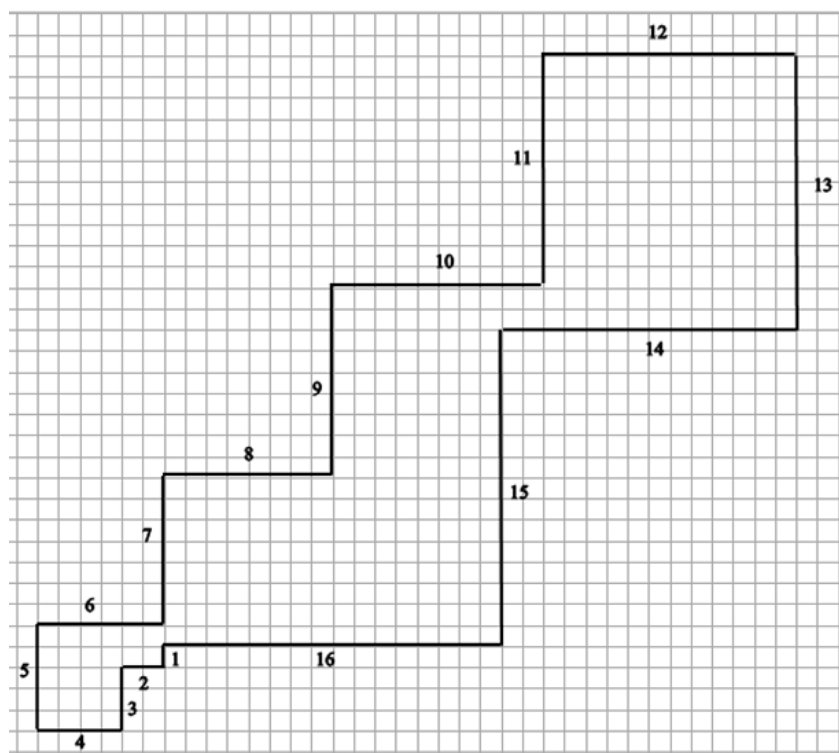
2. malu garumus sadala pa kopām:

- $H = \{2, 4, \dots, 8k\}$
- $V = \{1, 3, \dots, 8k - 1\}$
- $H_1 = \{2k + 2, 2k + 4, \dots, 2k + 4k\}$
- $H_2 = H \setminus H_1$
- $V_1 = \{2k + 1, 2k + 3, \dots, 6k - 1\}$
- $V_2 = V \setminus V_1$

3. zīmē perfektu polimino:

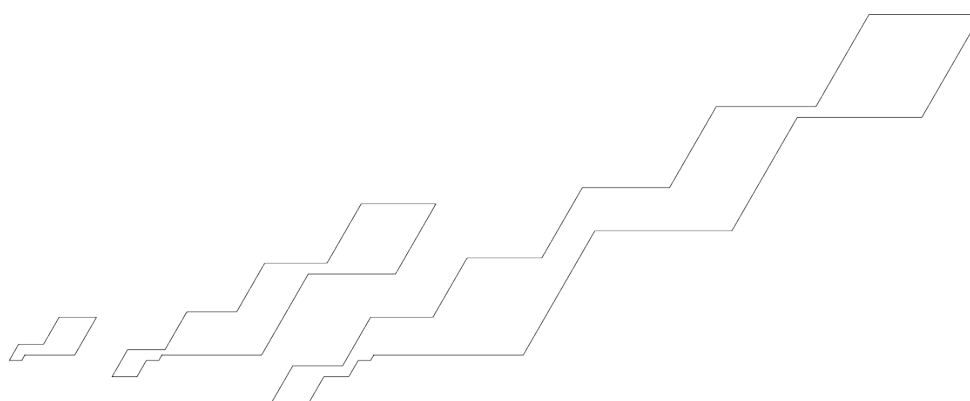
- malas ar garumiem no kopas H_1 liek uz rietumiem;
- malas ar garumiem no kopas H_2 liek uz austrumiem;
- malas ar garumiem no kopas V_1 liek uz dienvidiem;
- malas ar garumiem no kopas V_2 liek uz ziemeļiem.

2. attēlā ir attēlots 16-stūru perfekts polimino, veidots pēc dotās shēmas.

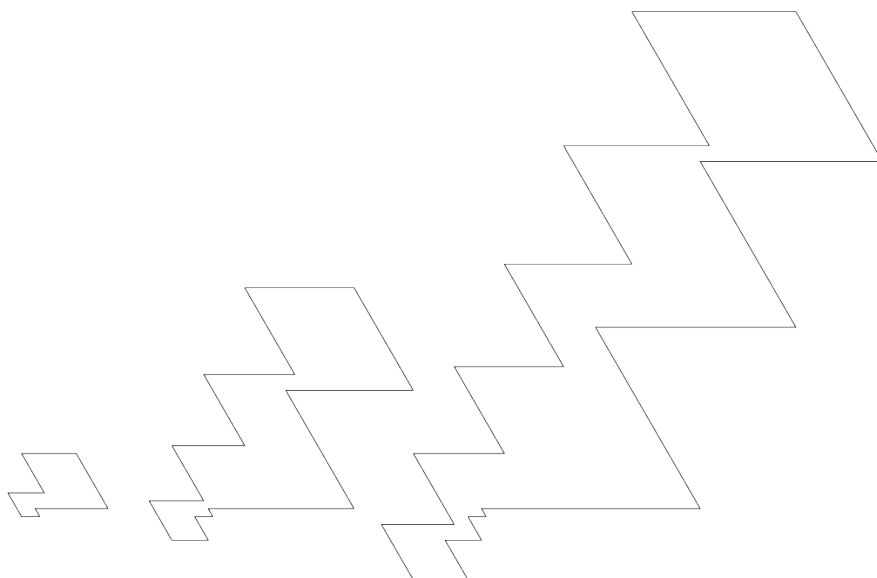


2. att. 16-stūru perfekts polimino

Tā kā katru polimino var reducēt uz diviem dažādiem polimondiem, visas vertikālās malas pagriežot par 60° pa labi, otrā gadījumā visas vertikālās malas pagriežot par 60° pa kreisi, ir iegūtas divas konstrukcijas $8k$ -stūru perfektiem polimondiem, kur $k \geq 1$. Šo konstrukciju pirmie polimondi ir attēloti 3. un 4. attēlā.

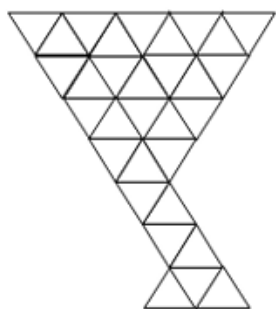


3. att. $8k$ -stūru perfektie polimondi, kur $k \geq 1$

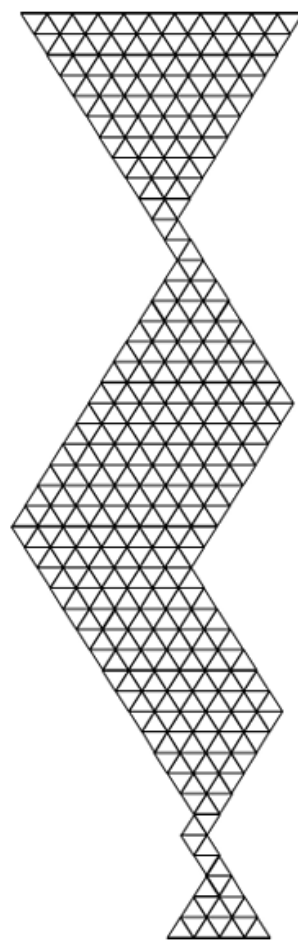


4. att. $8k$ -stūru perfektie polimondi, kur $k \geq 1$

E. Buliņas maģistra darbā [bulicna2018magiskie] ir dota shēma, kā konstruēt $(8k-2)$ -stūru perfektu polimondū, kur $k \geq 1$. 5. un 6. attēlā ir šīs konstrukcijas polimondi, kad k ir attiecīgi 1 un 2.



5. att. 6-stūru perfektie polimonds

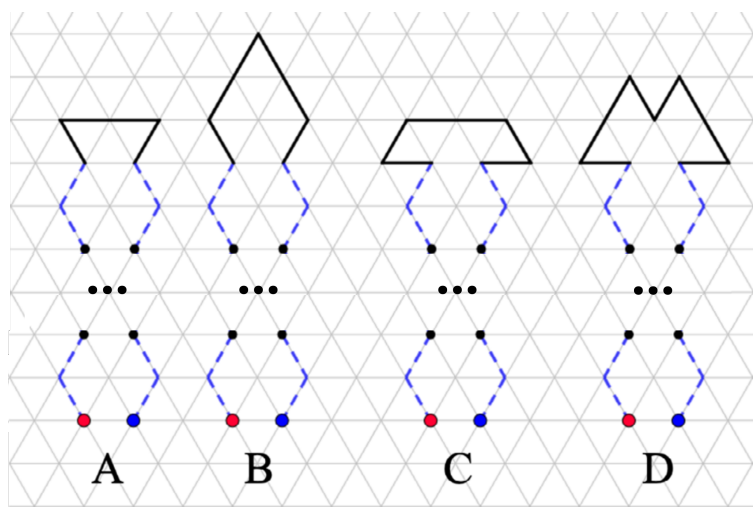


6. att. 14-stūru perfektie polimonds

2022. gada ATEE ikgadējās konferences “Būt vai nebūt lieliskam izglītotājam” (angl. *To be or not to be a great educator*) rakstu krājumā [cibulis_bulina] Elīna Buliņa ar Andreju Cibuli ir parādījuši perfektu polimondu eksistenci katram pāra malu skaitam $n \geq 6$.

Polimondi tiek iedalīti četrās kategorijās: $n = 6 + 8k$, $n = 8 + 8k$, $n = 10 + 8k$ un $n = 12 + 8k$, kur $k \geq 0$. Polimonds tiek veidots no divām daļām, 7. attēlā var redzēt daļu veidošanas ideju. Lai izveidotu perfektu polimondu $n = 6 + 8k$ malu skaitam, veido divas A veida laužas līnijas, priekš perfektiem polimondi ar $n = 8 + 8k$, $n = 10 + 8k$ un $n = 12 + 8k$ malu skaitu veido attiecīgi B, C un D veida laužas līnijas. Katrā no laužtajām līnijām jābūt $\frac{n}{2}$ nogriežņiem, tāpēc, skatoties uz 7. attēlu, katra veida laužtajai līnijai ir jāsaturs $4k$ raustītos, zilā krāsā iekrāsotos nogriežņus. Beigās šāda veida divas laužtās līnijas savieno kopā līniju galapunktos un, sākot no viena savienojuma punkta, sāk zīmēt malas secīgi ar garumiem no 1 līdz n . Šī metode izmanto invariantu īpašību, galapunkti nemainās, ja katru no malām vienādi daudz pagarina.

5. attēlā var redzēt, kā izskatīsies 6-stūru perfekts polimonds, bet 6. attēlā – 14-stūru perfekts polimonds.



7. att. Pāra skaita malu polimondus veidošana

Ar datorprogrammas palīdzību līdz malu skaitam 23 tika saskaitīts perfekto polimondus skaits. Rezultāti ir parādīti 1. tabulā. Priekš lielākām n vērtībām datorprogramma vairs netika darbināta, jo aprēķini prasītu pārāk daudz resursu.

Šī datorprogramma izmanto rekursīvu atkāpšanos (angl. *backtracking*), lai izveidotu visus polimondus dotam malu skaitam; sk. GitHub projektu. Lieliem n n -stūru perfekto polimondus pārlase ar jebkuru metodi kļūst lēna, jo to skaits strauji pieaug. “Backtracking” algoritmu var daudz kur izmantot, piemēram, sudoku mīklu vai dažādu šaha problēmu risināšanā. Pielikumā ir

pievienoti divi datorprogrammu kodi. ?? pielikumā ir kods, kas, izmantojot ?? pielikumā esošo “backtracking” algoritmu, atrod visus perfektos polimondus dotam malu skaitam n .

1. tabula

Perfekto polimondu skaits atkarībā no malu skaita

n	5	6	7	8	9	10	11	12	13	14
Skaits	1	1	2	9	3	4	21	87	288	477
n	15	16	17	18	19	20	21	22	23	
Skaits	1335	4026	8599	22326	67068	193445	526694	1464978	4284617	

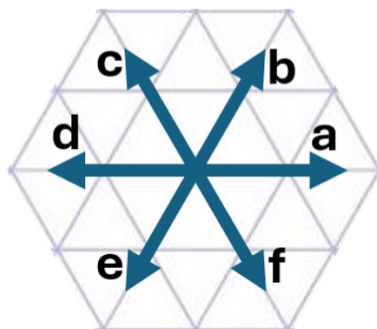
2. BEZKONTEKSTA GRAMATIKAS

Rakstā [cfg_def] bezkonteksta gramatika (angl. *context-free grammar*, CFG) G ir definēta kā četru elementu kopa: $G = (V, T, P, S)$, kur V ir netermināļu (angl. *non-terminals*) kopa, T ir termināļu (angl. *terminals*) kopa, P ir nosacījumu (angl. *rules*) kopa un S ir sākuma simbols. Valodas, kuras ir ģenerētas no bezkonteksta gramatikām, sauc par bezkonteksta valodām (angl. *context-free languages*, CFL).

Termināļi ir simboli, kas veido valodas elementus, un netermināļi palīdz izveidot šos elementus. Nosacījumi sastāv no trīs daļām: viena netermināļa, simbola “ $\rightarrow$ ”, nulles elementa vai termināļu un netermināļu kombinācijas. Netermināli aizstāj ar kombināciju, kas seko pēc bultas.

2.1. Polimundu iegūšana ar bezkonteksta gramatiku

Lai aprakstītu perfektu polimundu, var izmantot koordinātu sistēmu, kas attēlota 8. attēlā. Tā kā var pieņemt, ka polimonds sākas ar garāko malu un turpinās dilstošā secībā, tad jebkuru perfektu polimundu var aprakstīt ar burtiem a, b, c, d, e, f vienā vienīgā veidā, kā arī katrai burtu virknei atbilst viens un tas pats polimonds. Šie seši burti veido termināļu kopu $T = \{a, b, c, d, e, f\}$ bezkonteksta gramatikai, kas veido perfektus polimondus.



8. att. Koordinātu sistēma trijstūru režģī

Rakstā [cibulis_bulina] dotās perfekto polimundu konstrukcijas var aprakstīt ar bezkonteksta gramatikām. Piemēram, konstrukciju A, kura veido perfektu polimundu ar malu skaitu $n = 8k + 6$, kur $k \geq 0$, var aprakstīt šādi:

$S \rightarrow Pfdb$

$P \rightarrow cbc b P f e f e$

$P \rightarrow cae$

Šajā gadījumā P ir gramatikas neterminālis, kuru var aizstāt ar $cbcb P f e f e$ vai cae . Šie divi

likumi veido nosacījumu kopu P . Bezkonteksta valodas elementi tiek veidoti, sākot ar sākuma simbolu S , pēc tam tiek pielietoti nosacījumi. Brīdī, kad virkne vairs nesatur nevienu netermināli, ir iegūts elements, kas pieder bezkonteksta valodai, kas ģenerēta ar konkrēto bezkonteksta gramatiku.

Ja pirmajā izvēlē tiek pielietots pēdējais likums, iegūst burtu virni $caefdb$. Ja polimonds tiek veidots, sākot ar garāko malu, pēc tam malu garumus samazinot, pēc 8. attēla apzīmējumiem, šī burtu virkne atbilst polimondam 5. attēlā.

Ja pirmajā izvēlē tomēr sākumā pielieto pirmspēdējo likumu un tikai pēc tam pēdējo likumu, tad iegūst burtu $cbcbcaefefe$. Šī virkne apraksta 14-stūru perfektu polimondu, kurš ir attēlots 6. attēlā ($n = 8k + 6$, kur $k = 1$). Tomēr no šī brīža ir nepieciešams pieminēt, ka polimonds jāsāk veidot no malas ar garumu $2k$ dilstošā secībā (pēc malas ar garumu 1 vienība, jāturpina ar malu garumā n un tālāk atkal jāturpina dilstoši). Šis fakts ir jāatceras arī veidojot visus pārējos polimondus no virknēm, kas iegūtas ar doto bezkonteksta gramatiku, kā arī rakstā [**cibulis_bulina**] dotajām konstrukcijām B, C un D, izveidojot bezkonteksta gramatikas, perfektais polimonds arī jāsāk veidot no malas ar garumu $2k$, turpinot dilstošā secībā. Šādi izskatās bezkonteksta gramatikas konstrukcijām B, C un D:

Konstrukcija B	Konstrukcija C	Konstrukcija D
$n = 8k + 8, k \geq 0$	$n = 8k + 10, k \geq 1$	$n = 8k + 12, k \geq 1$
$S \rightarrow P f e c b$	$S \rightarrow P a e d c a$	$S \rightarrow P a e c e c a$
$P \rightarrow c b c b P f e f e$	$P \rightarrow c b c b P f e f e$	$P \rightarrow c b c b P f e f e$
$P \rightarrow c b f e$	$P \rightarrow d b a f d$	$P \rightarrow d b f b f d$

2024.gadā Kalvis Apsītis diskretās matemātikas konferencē prezentēja trīs konstrukcijas, kuras arī var aprakstīt ar bezkonteksta gramatikām. Šīs gramatikas veido termināļu virknes, kuras veido polimondus, sākot no garākās malas un turpinot dilstošā secībā. Konstrukcija, kas veido perfektu polimondus malu garumam $n = 8k + 1$, kur $k \geq 2$, izskatās šādi:

$S \rightarrow a c e c P d b$

$P \rightarrow e c e a P a b c b$

$P \rightarrow e a f$

9. att. $(8k + 1)$ -stūru perfektie polimondi, kur $k \geq 2$

3. PARALĒLĀS BEZKONTEKSTA GRAMATIKAS

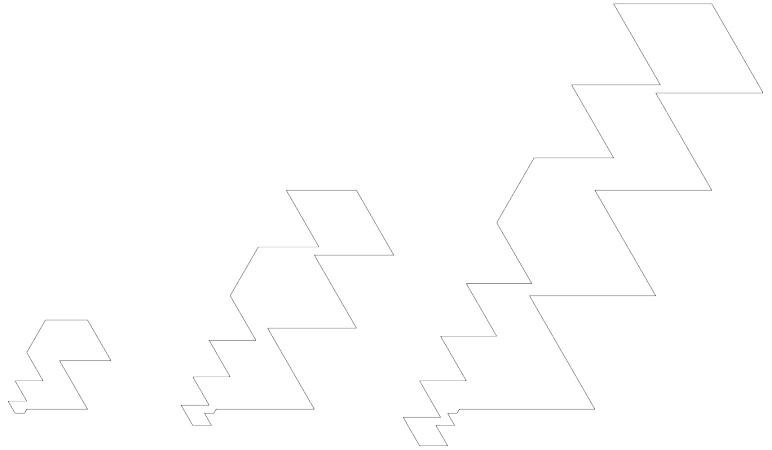
Rakstā [pcg_def] ir definēta paralēlā bezkonteksta gramatika (angl. *parallel context-free grammar*, PCG), izmantojot bezkonteksta gramatiku $G = (V, T, P, S)$. G ir paralēlā bezkonteksta gramatika, ja $\forall w_1 \xRightarrow{G} w_2$ un $\forall A \in V$, kur $w_1 = x_1 A x_2 A \dots A x_r$, $w_2 = x_1 \alpha x_2 \alpha \dots \alpha x_r$, $r \geq 1$, $A \rightarrow \alpha \in P$ un $x_1, \dots, x_r$ nesatur A . Šī definīcija pasaka, ka visi virknes saturošie netermināli ir vienlaikus jāaizstāj ar vieniem un tiem pašiem nosacījumiem. Valodu $L = L(G)$ sauc par paralēlo bezkonteksta valodu (angl. *parallel context-free language*, PCL), ja L ir ģenerēta ar paralēlo bezkonteksta gramatiku G .

Paralēlā bezkonteksta gramatikā var saskatīt līdzības ar Lindenmaiera sistēmām (angl. *Lindenmayer systems*) jeb L-sistēmām. Vienīgi L-sistēmās malu skaits parasti aug eksponenciāli, pazīstams piemērs ir Koha sniegpārsla.

3.1. Polimondū iegūšana ar paralēlo bezkonteksta gramatiku

Detalizēti aplūkosim $(8k - 3)$ -stūru perfekto polimondū konstrukciju, kura ir aprakstīta ar šādu paralēlo bezkonteksta gramatiku:

$S \rightarrow a\mathbf{P}_1 \text{ } cacd \mathbf{P}_2 \text{ } e \mathbf{P}_2 \text{ } fd f d f a \mathbf{P}_1 \text{ } b$
 $\mathbf{P}_1 \rightarrow ca \mathbf{P}_1$
 $\mathbf{P}_2 \rightarrow fd \mathbf{P}_2$



10. att. $(8k - 3)$ -stūru perfektie polimondi, kur $k \geq 2$

Šajā gadījumā nosacījumu kopa ir $P = \{\mathbf{P}_1, \mathbf{P}_2\}$. Ir divas iespējas, ko ar sākuma virkni (sauc arī par aksiomu) var darīt – izdzēst no tās visus neterminālus vai arī katru netermināli aizstāt ar nosacījumos norādīto terminālu un neterminālu kombināciju (vienlaikus ir jāaizstāj katrs neterminālis).

Ja pirmajā izvēlē izvēlas izdzēst neterminālus, tad iegūst terminālu virkni $acacdefdfdfab$. Šajā un visās nākamajās konstrukcijās pieņemam, ka polimonds vienmēr tiks veidots, sākot ar

garāko malu, pēc tam malu garumus samazinot. Tādā gadījumā pēc 8. attēla apzīmējumiem, šī burtu virkne atbilst tieši mazākajam polimondam ?? attēlā.

Ja pirmajā izvēlē tomēr izvēlas izpildīt nosacījumus, tad iegūst virkni: $acacacP_1df dP_2efdfP_2dfdfacaP_1b$. Tagad atkal var izvēlēties jebkuru no iepriekš minētajām divām darbībām. Ja izdzēš visus netermināļus, tad iegūst vidējo polimondus ?? attēlā. Var arī turpināt pielietot nosacījumus neierobežoti ilgi. Ja n reizes tiek izvēlēts katru virknes netermināli aizstāt ar nosacījumos norādīto termināļu un netermināļu kombināciju, tad beigās, no virknes izdzēšot visus četrus netermināļus, tiks iegūts perfekts polimonds ar malu skaitu $8(n + 2) - 3$.

3.2. Konstrukciju korektums

Paralēlās bezkonteksta gramatikas pielietojums iespējams ir viskompaktākais veids, kas ļoti ātri un viegli iedod burtu virkni perfektam $(8k - 3)$ -stūru polimondam, kur $k \geq 2$. Tomēr ir arī citi veidi, kā aprakstīt šo konstrukciju. Apskatīsim vienu no šiem veidiem, jo tas palīdzēs pēc tam viegli pamatot, ka šī konstrukcija vienmēr dos polimondus, kuri beigās atgriezīsies tajā pašā punktā, kur sākās.

?? attēlā attēlotos polimondus un visus nākamos šīs pašas konstrukcijas polimondus var iegūt arī pēc šādas shēmas:

1. izvēlas $k \geq 2$,
2. malas ar garumiem no $8k - 3$ līdz $6k - 2$ pēc kārtas liek uz $a, c, a, c, \dots$,
3. malas ar garumiem no $6k - 3$ līdz $4k + 1$ pēc kārtas liek uz $d, f, d, f, \dots$,
4. malu ar garumu $4k$ liek e virzienā,
5. malas ar garumiem no $4k - 1$ līdz $2k - 1$ pēc kārtas liek uz $f, d, f, d, \dots$,
6. malu ar garumu $2k - 2$ liek a virzienā,
7. malas ar garumiem no $2k - 3$ līdz 2 , pēc kārtas liek uz $c, a, c, a \dots$,
8. malu ar garumu 1 vienība liek b virzienā.

Tagad pierādīsim, ka polimonds beigās atgriezīsies sākumpunktā.

Pierādījums. Ja b un c virzienā likto malu garumu summa būs vienāda ar e un f virzienā likto

malu garumu summu, tad sanāk, ka tik vienības, cik polimonds ir pagājis uz augšu, tikpat tas ir pagājis uz leju.

Ja a virzienā likto malu garumu summa un b un f virzienā likto malu garumu pussumma kopā būs vienāda ar d virzienā likto malu garumu summu un c un e virzienā likto malu garumu pussummu kopā, tad ir zināms, ka tik vienības, cik polimonds ir pagājis pa labi, tikpat tas ir pagājis pa kreisi.

Lai aprēķinātu uz katru virzienu likto malu garumu summu, izmantosim divas zināmas formulas:

$$1 + 3 + 5 + \dots + (2n - 1) = n^2, \quad (3.1)$$

$$2 + 4 + 6 + \dots + 2n = n(n + 1). \quad (3.2)$$

No šīm formulām varam iegūt šādas formulas:

$$(2m - 1) + (2m + 1) + \dots + (2n - 3) + (2n - 1) = n^2 - (m - 1)^2, \quad (3.3)$$

$$2m + (2m + 2) + \dots + (2n - 2) + 2n = n(n + 1) - m(m - 1). \quad (3.4)$$

Virzienā a tiek liktas nepāra garuma malas ar garumiem no $8k - 3$ līdz $6k - 1$. Pēc formulas (??) varam aprēķināt šo malu garumu summu:

$$(4k - 1)^2 - (3k - 1)^2 = 7k^2 - 2k.$$

Vēl virzienā a tiek liktas pāra garuma malas ar garumiem no 2 līdz $2k - 2$. Pēc formulas (??) varam aprēķināt šo malu garumu summu:

$$(k - 1)k = k^2 - k.$$

Kopā virzienā a likto malu garumu summa ir:

$$7k^2 - 2k + k^2 - k = 8k^2 - 3k.$$

Tieši tādā pašā veidā, skatoties uz shēmu un formulām, varam aprēķināt visos pārējos virzienos likto malu garumu summas. b virzienā tiek likta tikai viena mala, kuras garums ir 1 vienība,

c virzienā likto malu garumu summa sanāk $8k^2 - 5k$, d virzienā: $8k^2 - 7k + 1$, e virzienā tiek likta tikai viena mala, kuras garums ir $4k$, bet f virzienā likto malu garumu summa ir $8k^2 - 9k + 1$.

Saskaitot kopā b un c virzienā likto malu garumu summas, iegūst $8k^2 - 5k + 1$. Saskaitot kopā e un f virzienā likto malu garumu summas, arī iegūst $8k^2 - 5k + 1$. Tas nozīmē, ka tik, cik tika paiets uz augšu, tikpat arī tika paiets uz leju, kas nozīmē, ka mala ar garumu 1 vertikāli beidzas turpat, kur sākas garākā mala.

Saskaitot kopā b un f virzienā likto malu garumu pussummu ar a virzienā likto malu garumiem, iegūst $12k^2 - 7.5k + 1$. Tādu pašu summu iegūst arī saskaitot c un e virzienā likto malu garumu pussummu ar d virzienā likto malu garumiem. Tas nozīmē, ka tik, cik tika paiets pa labi, tikpat tika paiets pa kreisi, kas nozīmē, ka mala ar garumu 1 horizontāli beidzas turpat, kur sākas mala ar garumu $8k - 3$.

Tā kā mala ar garumu 1 gan horizontāli, gan vertikāli beidzas turpat, kur sākas mala ar garumu $8k - 3$, tad konstruējot polimondus, iegūst slēgtu figūru. $\square$

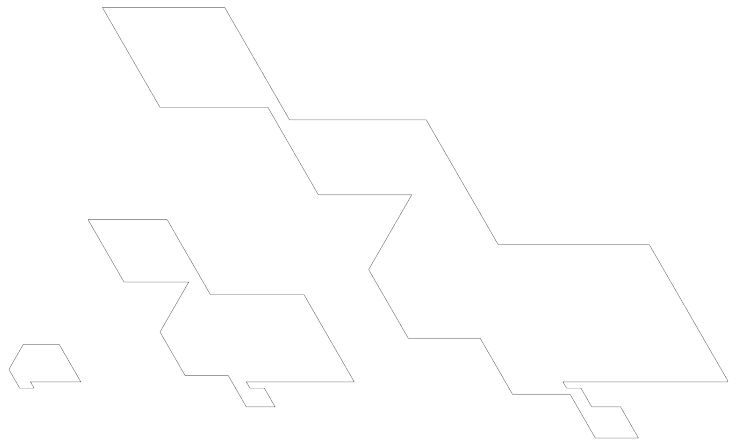
3.3. Pārējās konstrukcijas

Tieši tādā pašā veidā, kā iepriekšējā sadaļā, var pierādīt arī visām pārējām 13 atrastajām konstrukcijām, ka tās vienmēr dos slēgtas figūras.

Lai apgalvotu, ka šīs konstrukcijas tiešām ir korektas un dod perfektus polimondus, vajadzētu arī pierādīt, ka tās sev nekad nepieskaras un sevi nekrusto. Šis pagaidām vēl nav pierādīts. Tomēr vairākām konstrukcijām, vien paskatoties uz pirmajiem perfektajiem polimondiem, uzreiz ir skaidrs, ka polimondi sevi nekad nekrustos un sev nekad nepieskarsies (sk., piemēram, ??., ??., ?? un ?? attēlu).

Aksioma: $aS_1cdS_2efaS_2S_1c$

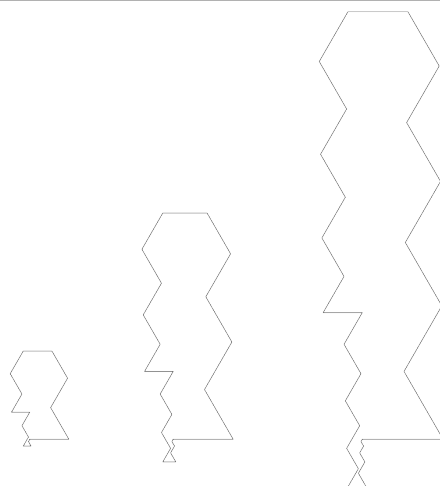
Nosacījumi: $\{ S_1 \rightarrow cdS_1, S_2 \rightarrow faS_2 \}$



11. att. $(8k - 1)$ -stūru perfektie polimondi, kur $k \geq 1$

Aksioma: $aS_1cbcdfeS_2aeS_2aS_1cb$

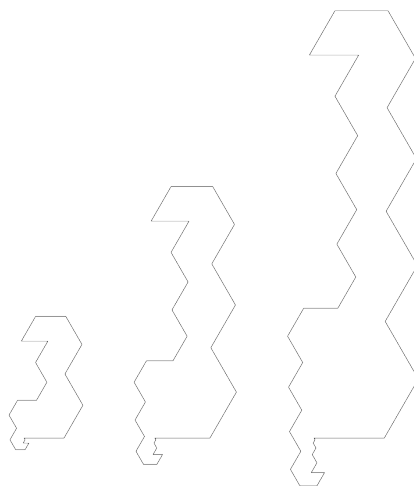
Nosacījumi: $\{ S_1 \rightarrow cbS_1,$
 $S_2 \rightarrow feS_2 \}$



12. att. $(8k - 1)$ -stūru perfektie polimondi, kur $k \geq 2$

Aksioma: $aS_1bcbcdcaS_2efedS_2efefabdS_1bc$

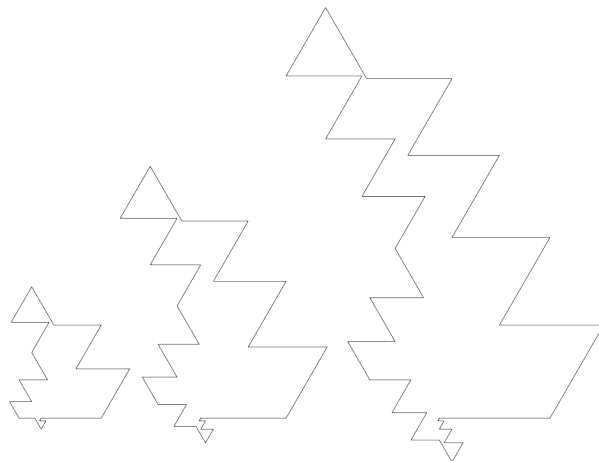
Nosacījumi: $\{ S_1 \rightarrow bcS_1,$
 $S_2 \rightarrow efS_2 \}$



13. att. $(8k - 3)$ -stūru perfektie polimondi, kur $k \geq 3$

Aksioma: $aS_1bdbdceaS_2efdfdfaS_2fS_1bdb$

Nosacījumi: $\{ S_1 \rightarrow bdS_1,$
 $S_2 \rightarrow eaS_2 \}$



14. att. $(8k - 5)$ -stūru perfektie polimondi, kur $k \geq 3$

Aksioma: $aS_1bcd fS_2edS_2efacS_1$

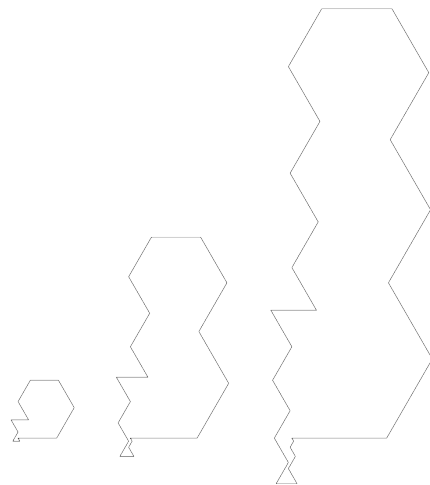
Nosacījumi: $\{S_1 \rightarrow bcS_1,$
 $S_2 \rightarrow efS_2\}$



15. att. $(8k - 5)$ -stūru perfektie polimondi, kur $k \geq 2$

Aksioma: $aS_1bcdeS_2fdS_2feacS_1$

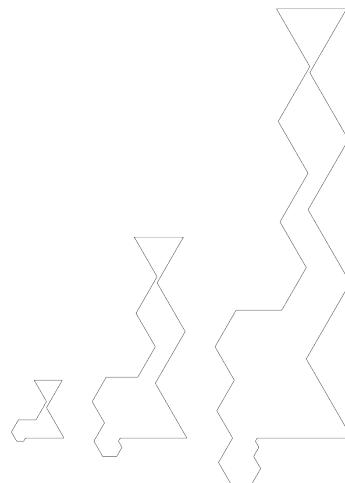
Nosacījumi: $\{S_1 \rightarrow bcS_1,$
 $S_2 \rightarrow feS_2\}$



16. att. $(8k - 5)$ -stūru perfektie polimondi, kur $k \geq 2$

Aksioma: $aS_1cbdfS_2edS_2efabS_1$

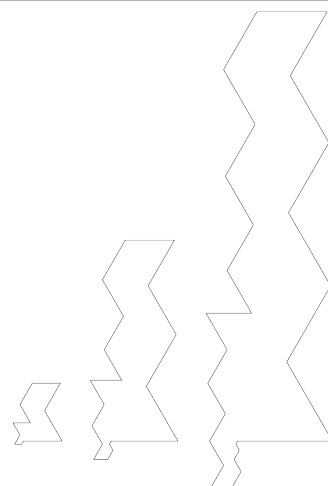
Nosacījumi: $\{S_1 \rightarrow cbS_1,$
 $S_2 \rightarrow efS_2\}$



17. att. $(8k - 5)$ -stūru perfektie polimondi, kur $k \geq 2$

Aksioma: $aS_1cbdeS_2fdS_2feabS_1$

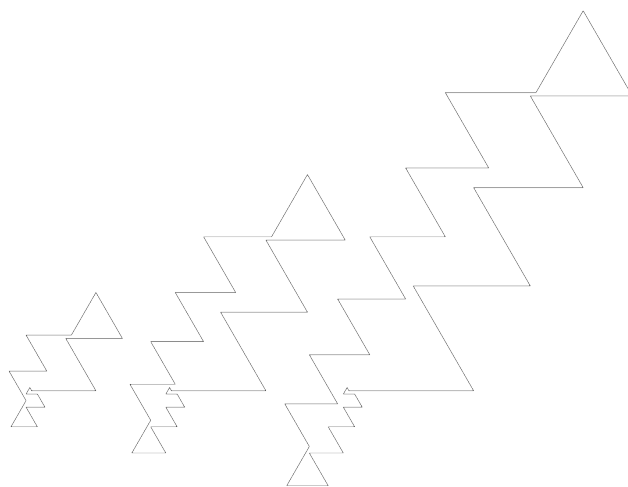
Nosacījumi: $\{ S_1 \rightarrow cbS_1,$
 $S_2 \rightarrow feS_2 \}$



18. att. $(8k - 5)$ -stūru perfektie polimondi, kur $k \geq 2$

Aksioma: $aS_1caceS_2dfdf eaS_1cacdbf$

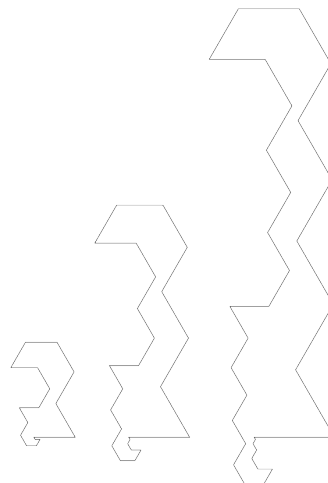
Nosacījumi: $\{ S_1 \rightarrow caS_1,$
 $S_2 \rightarrow dfdf S_2 \}$



19. att. $(8k - 7)$ -stūru perfektie polimondi, kur $k \geq 3$

Aksioma: $aS_1cbcdeaS_2fedS_2fefabdS_1c$

Nosacījumi: $\{ S_1 \rightarrow cbS_1,$
 $S_2 \rightarrow feS_2 \}$



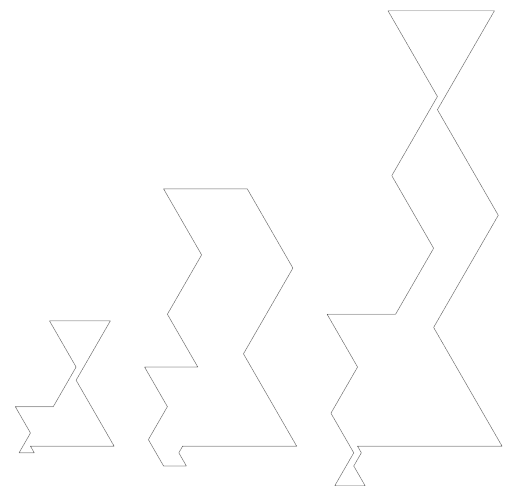
20. att. $(8k - 7)$ -stūru perfektie polimondi, kur $k \geq 3$

Šīs ir visas līdz šim atrastās konstrukcijas ar indukcijas soli 8. Tomēr pietiek arī ar tikai četrām konstrukcijām, pa vienai katrā kategorijā $(8k - 1, 8k - 3, 8k - 5$ un $8k - 7)$, lai būtu veids, kā konstruēt perfektu polimondus jebkuram nepāra malu skaitam.

Vēlāk tika atrastas arī konstrukcijas ar indukcijas soli 4, kas nozīmē, ka pietiek vien ar divām konstrukcijām, lai spētu uzkonstruēt polimondus katram nepāra malu skaitam (sākot no 17). Zemāk ir parādītas šo konstrukciju shēmas un šo konstrukciju pirmie trīs polimondi.

Aksioma: $acbS_1dfeS_2dfeS_2acS_3$

Nosacījumi: $\{ S_1 \rightarrow cT_1,$
 $S_2 \rightarrow fT_2,$
 $S_3 \rightarrow bT_3,$
 $T_1 \rightarrow bS_1,$
 $T_2 \rightarrow eS_2,$
 $T_3 \rightarrow cS_3 \}$



21. att. $(4k - 1)$ -stūru perfektie polimondi, kur $k \geq 3$

Aksioma: $abcS_1defS_2defS_2abS_3$

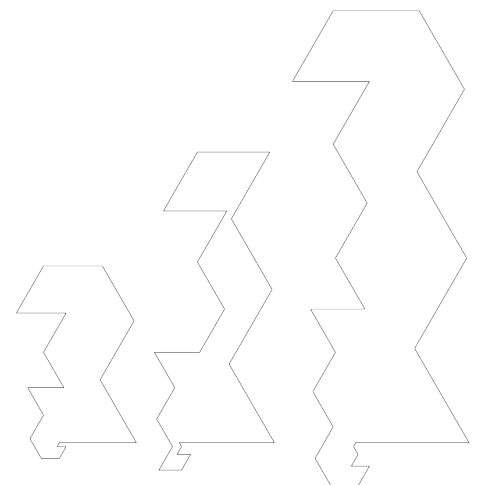
Nosacījumi: $\{ S_1 \rightarrow bT_1,$
 $S_2 \rightarrow eT_2,$
 $S_3 \rightarrow cT_3,$
 $T_1 \rightarrow cS_1,$
 $T_2 \rightarrow fS_2,$
 $T_3 \rightarrow bS_3 \}$



22. att. $(4k - 1)$ -stūru perfektie polimondi, kur $k \geq 3$

Aksioma: $acbcS_1deaeS_2dfeS_2abdbS_3$

Nosacījumi: $\{ S_1 \rightarrow bT_1,$
 $S_2 \rightarrow eT_2,$
 $S_3 \rightarrow cT_3,$
 $T_1 \rightarrow cS_1,$
 $T_2 \rightarrow fS_2,$
 $T_3 \rightarrow bS_3 \}$



23. att. $(4k - 3)$ -stūru perfektie polimondi, kur $k \geq 5$

SECINĀJUMI

Kursa darbā aplūkotie perfektie polimondi ir diezgan maz apskatīta tēma, tomēr perfektos polimondus ir vērts pētīt, jo ar viņiem var formulēt dažādus olimpiāžu, kā arī kombinatorikas un programmēšanas uzdevumus.

Ir atrastas vairākas dažādas metodes, kas veido perfektus polimondus jebkuram malu skaitam – gan pāra, gan nepāra. Lai gan visas šīs metodes, konstrukcijas līdz šim vienmēr ir devušas korektus polimondus, nav pilnībā pierādīts, ka tās tiešām ir pareizas, nav noliegts apgalvojums, ka kaut kad (varbūt bezgalīgi lielam malu skaitam) izveidosies polimonds, kas posmos sev pieskarsies vai sevi krustos, vai sevi pārklās.

Iespējams, varētu arī atrast daudz dažādu sakarību, ja nedaudz paplašinātu perfekto polimondus klasi un apskatītu izogonus, tomēr tikai tos, kuri sevi nekrusto (drīkst pārklāties un pieskarties posmos). Šī izogonu apakšklase līdz šim vēl nav tikusi definēta un pētīta.

PATEICĪBAS

- Latvijas Universitātes Eksakto zinātņu un tehnoloģiju fakultātes docentam K. Apsītim par labojumiem un sniegto palīdzību darba izstrādē, datorprogrammu izveidē, kā arī konstrukciju aprakstīšanā.
- Kursa darba vadītājam – profesoram A. Cibulim par palīdzību tēmas izvēlē un labojumiem.

PIELIKUMI

A. “BACKTRACKING” ALGORITHMS

```
1
2 class Backtrack:
3
4     # Backtracker objekts(nepabeigts šaha galdiņš, polimonds
5     # vai kas cits).
6     b = None
7
8     # Konstruktors tikai iekopē padoto objektu
9     def __init__(self, b):
10         self.b = b
11
12     # Mēģina risināt uzdevumu, atrodoties koka līmenī "level".
13     # level=0 ir pašā augšā - pirms izdarīti "gājieni".
14     def attempt(self, level):
15         successful = False
16         moveIterator = self.b.moves(level)
17
18         for move in moveIterator:
19             successful = False
20             if self.b.valid(level, move):
21                 self.b.record(level, move)
22                 if self.b.done(level):
23                     successful = True
24                 else:
25                     successful = self.attempt(level+1)
26                 if not successful:
27                     self.b.undo(level, move)
28             if successful:
29                 self.b.display()
30                 self.b.undo(level, move)
```

B. POLIMONDU PROBLĒMAS KODS

```

1 from backtrackk import *
2
3 class Polimondi:
4     liss = []
5     mysett = []
6
7     def __init__(self, n):
8         self.N = n
9         self.liss.append([0, self.N])
10        for i in range(1, self.N+1):
11            self.mysett.append([i, 0, -i])
12
13        # Funkcija 22.rindā pārbauda, vai nogrieznis ar garumu 1
14        # nav novietots horizontāli.
15        # Visa pārējā funkcija sataisa jaunu sarakstu (new_mysett_elem),
16        # kur ir ieliktas iespējamās jauno malu koordinātas.
17        # 47. rinda pārbauda, vai neviens no jaunajiem punktiem
18        # jau neatrodas sarakstā mysett.
19        def checks(self, move, level):
20            self.level = level
21            self.move = move
22            if move[0] == 0 and (move[1] == 1 or move[1] == -1):
23                return False
24            new_mysett_elem = []
25            ped = self.mysett[-1]
26            l10 = self.move[0]
27            l11 = self.move[1]
28            for k in range(self.N-self.level):
29                if l10 == 0 and l11 > 0:

```

```

30         new_mysett_elem.append(
31             [(k+1+ped[0]), (ped[1]), (ped[2]-(k+1))])
32     elif ll0 == 0 and ll1 < 0:
33         new_mysett_elem.append(
34             [(ped[0]-(k+1)), (ped[1]), (k+1+ped[2])])
35     elif ll0 > 0 and ll1 > 0:
36         new_mysett_elem.append(
37             [(k+1+ped[0]), (ped[1])-(k+1), (ped[2])])
38     elif ll0 < 0 and ll1 < 0:
39         new_mysett_elem.append(
40             [(ped[0])-(k+1), (k+1+ped[1]), (ped[2])])
41     elif ll0 < 0 and ll1 > 0:
42         new_mysett_elem.append(
43             [(ped[0]), (k+1+ped[1]), (ped[2])-(k+1)])
44     elif ll0 > 0 and ll1 < 0:
45         new_mysett_elem.append(
46             [(ped[0]), (ped[1])-(k+1), (k+1+ped[2])])
47     if all(a not in self.mysett for a in new_mysett_elem) == False:
48         return False
49     else:
50         self.new_mysett_elem = new_mysett_elem
51         # print(self.liss)
52         # print(self.new_mysett_elem)
53         # print(self.mysett)
54         # print(" ")
55         return True
56
57     # Funkcija pārbauda, vai pievienojot jauno malu, ir iespējams
58     # atgriezties atpakaļ sākumpunktā ar atlikušajām malām.
59     # d-jau izveidotā polimonda augstums, v-platums.
60     def checks2(self, move, level):
61         self.level = level
62         self.move = move

```

```

63     self.list1 = self.liss.copy()
64     self.list1.append(move)
65     d = 0
66     v = 0
67     for k in self.list1:
68         d += k[0]
69         v += k[1]
70     m = self.N - self.level - 1
71     mm = (m*(m+1))/2
72     # self.debugstate2(mm, v, d, m, self.level)
73     if (mm < abs(d)) or (mm < abs(v)):
74         return False
75     return True
76
77 def valid(self, level, move):
78     self.p = move
79     # self.debugstate(move, level)
80     c1 = self.checks(move, level)
81     c2 = self.checks2(move, level)
82     # print("c1={}, c2={}".format(c1, c2))
83     return c1 and c2
84
85 def done(self, level):
86     # if level == self.N - 1:
87     #     return True
88     # return False
89     return (level == self.N - 1)
90
91 # Pievienojam jauno malu sarakstā liss un šīs malas visus punktus
92 # sarakstā mysett.
93 def record(self, level, move):
94     self.liss.append(move)
95     for k in self.new_mysett_elem:

```

```

96         self.mysett.append(k)
97
98     # Parāpjamies atpakaļ, ja bija sasniegts strupceļš.
99     def undo(self, level, move):
100         self.liss = self.liss[:-1]
101         self.mysett = self.mysett[:-(self.N-(level))]
102         self.new_mysett_elem = []
103         self.list1 = []
104
105     # Izvada risinājumu.
106     def display(self):
107         print("{}, ".format(self.liss))
108
109     # Atgriež četrus (pirmajā solī divus) iespējamus gājienus,
110     # kā polimondus no dotā punkta var turpināt.
111     def moves(self, level):
112         self.level = level
113         if self.level == 1:
114             x111 = self.N - 1
115             li1 = [[x111, -x111 / 2], [x111, x111 / 2]]
116         else:
117             x1 = self.N - self.level
118             p = self.p
119             if p[0] == 0 and p[1] > 0 or p[0] == 0 and p[1] < 0:
120                 li1 = [[x1, -x1 / 2], [x1, x1 / 2],
121                        [-x1, -x1 / 2], [-x1, x1 / 2]]
122             elif p[0] < 0 and p[1] > 0 or p[0] > 0 and p[1] < 0:
123                 li1 = [[0, -x1], [x1, x1 / 2], [0, x1], [-x1, -x1 / 2]]
124             elif p[0] < 0 and p[1] < 0 or p[0] > 0 and p[1] > 0:
125                 li1 = [[0, -x1], [x1, -x1 / 2], [0, x1], [-x1, x1 / 2]]
126         return li1
127
128     def main():

```

```
129     q = Polimondi(n)
130     b = Backtrack(q)
131     if b.attempt(1):
132         q.display()
133
134 if __name__ == '__main__':
135     main()
136
```